Bence senin hedeflerin düşünüldüğünde **Python'u sistemin ana shell dili yapmak stratejik bir hata olabilir**.

Senin anlattığın vizyon:

- UMFS (Universal Massive File System)
- Dünya ölçeğinde veri merkezi altyapısı
- GNU Assembler + C + C++
- Kendi AI frameworkün
- Kendi model eğitim altyapın
- Hazır framework bağımlılığını minimumda tutmak
- Maksimum performans ve kontrol

ise mimariyi şöyle düşünürdüm:

Katman 1: Çekirdek (Kernel Space Benzeri)

Burada:

- GNU Assembler
- C
- C++

olur.

Bütün kritik bileşenler:

- Storage Engine
- Block Device Layer
- Metadata Engine
- Cluster Manager
- Replication
- Distributed Lock Manager
- Scheduler
- Network Stack
- AI Runtime

C/C++.

Katman 2: Sistem Script Dili

Burada seçenekler:

Python

Avantaj:

- Çok kişi biliyor
- Hızlı geliştirme
- AI ekosistemi devasa

Dezavantaj:

- Runtime ağır
- GIL problemi
- Devasa bağımlılık zinciri
- CPython'a bağımlılık
- Uzun vadede "yabancı" hissettirir

UMFS gibi bir projede Python'un zamanla ikinci bir işletim sistemi haline gelme riski vardır.

Guile Scheme

GNU tarafında çok mantıklı.

Avantajlar:

- GNU projesinin resmi Lisp'i
- Embeddable
- Hafif
- Runtime küçük
- DSL geliştirmeye uygun
- Konfigürasyon dili olarak mükemmel

Mesela:

```
(cluster
  (name "eu-west")
  (replicas 5)
  (compression zstd))
```

gibi.

GNU ruhuna da uyuyor.

Common Lisp

En sevdiğim seçeneklerden biri.

Çünkü:

- Lisp makroları inanılmaz güçlü.
- Dilin kendisini genişletebilirsin.

- AI araştırmalarının ilk yıllarındaki en önemli dillerden biridir.

Ama:

- Ecosystem görece küçük.
- Öğrenme eğrisi yüksek.

Kendi Lisp'in

Eğer UMFS gerçekten Amassivus Datacenter Suite'in çekirdeği olacaksa ben bunu yapardım.

Mesela:

Amassivus Script

veya

UMFS Lisp

adıyla.

S-expression temelli:

```
(mount
 :cluster "europe"
 :replicas 5
 :compression "zstd")
```

Bunu C++ ile yazarsın.

Parser:

- Flex/Bison
- veya recursive descent parser

Runtime:

- Modern C++23

AI tarafında ne yapardım?

Python'u tamamen yasaklamazdım.

Şöyle:

```
UMFS Core
  |
  +-- libamassivus_ai.so
```

```
  |
  +-- C++ Runtime
  |
  +-- Amassivus Script
```

ve ayrıca:

```
bindings/
 ├── python
 ├── rust
 ├── go
 └── javascript
```

oluştururdum.

Yani:

- Çekirdek C++
- Eğitim altyapısı C++
- Tensor işlemleri C++
- GPU backend C++
- Python sadece binding

tıpkı:

- PyTorch
- TensorFlow
- OpenCV
- Qt

gibi.

Asıl motor C++, Python sadece dış arayüz.

Benim sana önerim

Mert, senin geçmişteki hedeflerini de düşününce (GNU, özgür yazılım, sistem programlama, kendi ekosistemini kurma arzun):

Guile Scheme + C++ kombinasyonu en mantıklı başlangıç noktası.

Uzun vadede ise:

1. UMFS Core → C++23
2. Performans kritik bölümler → C + GNU Assembler
3. Sistem otomasyonu → Guile Scheme

4. AI Engine → C++
5. Python → sadece isteğe bağlı binding katmanı

şeklinde bir mimari kurarsan hem GNU felsefesine yakın kalırsın hem de gelecekte kendi AI frameworkünü geliştirmek istediğinde Python'a bağımlı kalmazsın.

Hatta daha ileri gidersek, Emacs'taki gibi gömülü bir Lisp motoru kullanarak UMFS'nin tamamını script edilebilir hale getirebilir ve veri merkezi yöneticilerinin sistemi yeniden derlemeden davranışları değiştirmesini sağlayabilirsin. Bu yaklaşım, dünya ölçeğinde tasarladığın bir platform için uzun vadede Python'dan çok daha özgün ve sürdürülebilir bir mimari oluşturur.